

Ideation Phase

Brainstorm & Idea Prioritization Template

Date	30 September 2026
Team ID	SWTID-2026-9596
Project Name	Implement client script and UI policy
Maximum Marks	4 Marks

STEP-1: TEAM GATHERING, COLLABORATION AND SELECT THE PROBLEM STATEMENT

1. DEFINE YOUR PROBLEM STATEMENT

How might we statement: How might we eliminate manual data entry errors, stop slow page reloads during form submissions, and dynamically guide users while filling out IT Service Management forms (like Incidents or Change Requests) in ServiceNow?

2. SET THE GOAL

To build a highly responsive, browser-side automation layer within ServiceNow using JavaScript (Client Scripts) and conditional rules (UI Policies). This setup ensures instant data validation and makes fields visible or mandatory based on user choices without straining server performance.

STEP-2: BRAINSTORM, IDEA LISTING AND GROUPING

TEAM BRAINSTORMING NOTES (STICKY NOTES DATA):

- * Idea 1 (Frontend Optimization): Use Client Scripts to prevent users from selecting a past date or time in change schedules.
- * Idea 2 (Process Automation): Use a UI Policy to automatically show an "Urgent Business Justification" text area only when an incident priority is marked as Critical.
- * Idea 3 (Data Validation): Use an onSubmit Client Script to verify if a valid asset tag is attached before letting a user submit a hardware request.
- * Idea 4 (User Guidance): Use Client Scripts to throw real-time flash alerts or field messages when a VIP user profile is loaded on the form.
- * Idea 5 (Access Control): Use UI Policies to lock fields (make them read-only) once a ticket state moves to Closed or Resolved.

GROUPED IDEAS / CLUSTERS:

- * Cluster A: Form Validation & Security Rules (Focuses on checking data right before submission - Idea 1, Idea 3).
- * Cluster B: Dynamic UI Polish & Field Layouts (Focuses on hiding/showing fields cleanly on the screen - Idea 2, Idea 5).
- * Cluster C: VIP & Exception Handling Notifications (Focuses on immediate browser alerts - Idea 4).

STEP-3: IDEA PRIORITIZATION

1. HIGH IMPORTANCE & HIGH FEASIBILITY (Immediate Action / Best Solutions)

- * Selected Idea: Combined implementation of onChange UI Policies (to toggle critical fields dynamically) and onSubmit Client Scripts (for date and phone number validation).
- * Reason: UI Policies don't require code, making them extremely fast to deploy and maintain. Client Scripts cleanly handle complex data checks right in the browser.

2. HIGH IMPORTANCE & LOW FEASIBILITY (Long-term / Complex)

- * Idea: Creating advanced Client Scripts that execute heavy GlideRecord API scripts to pull third-party database details into the active browser window.
- * Reason: Running heavy database calls directly inside client-side scripts slows down form performance for the end-user.

3. LOW IMPORTANCE & HIGH FEASIBILITY (Quick Wins / Optional Features)

- * Idea: Writing local onLoad alerts just to greet a user or show their basic department details on static forms.
- * Reason: While it is easy to script, it adds very little functional value to business process workflows.

FINAL PROJECT CHOICE FOR IMPLEMENTATION:

We prioritize deploying UI Policies for conditional field behaviors (Visible/Mandatory/Read-only) and Client Scripts for advanced business rules validation (such as date sequencing and submission locks). This approach delivers an optimized interface with robust error checking.

STEP-4: SOFTWARE & HARDWARE REQUIREMENTS (SRS)

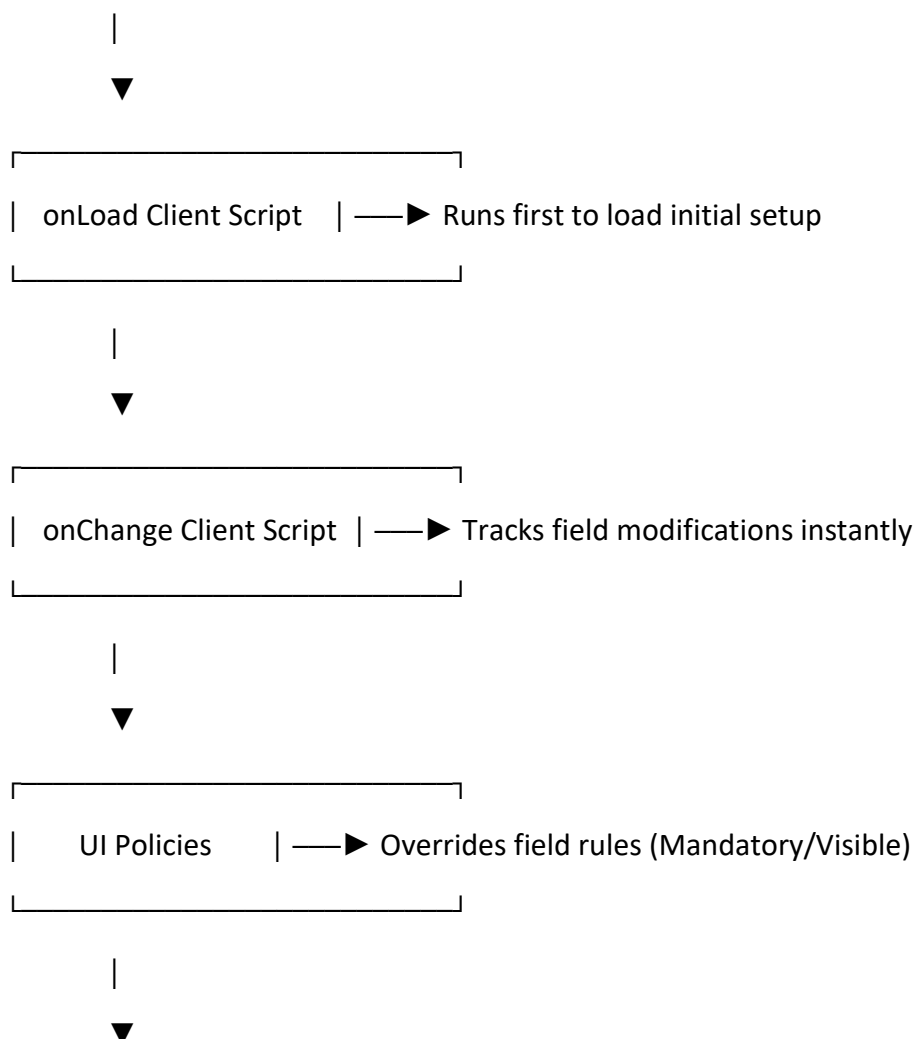
To deploy and test these client-side configurations, the following environments and dependencies are required:

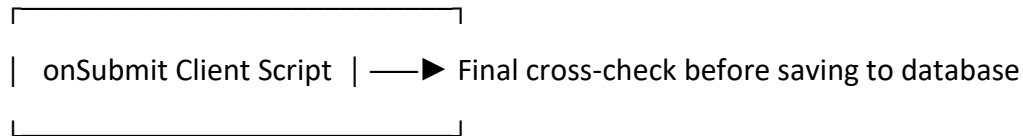
- * Platform Environment: ServiceNow Personal Developer Instance (PDI)
- * Development Language: JavaScript (ECMAScript 5 / ES6 compatible with ServiceNow APIs)
- * Client-Side Framework: AngularJS / GlideForm API (g_form) and GlideUser API (g_user)
- * Supported Browsers: Google Chrome (Latest), Mozilla Firefox, or Microsoft Edge
- * Hardware Requirement: Any standard development system with an active internet connection to access the ServiceNow cloud console.

STEP-5: PROJECT ARCHITECTURE & EXECUTION FLOW

=====

[User Opens / Interacts with ServiceNow Form]





STEP-6: SPECIFIC USE-CASE SPECIFICATIONS (FUNCTIONAL REQUIREMENTS)

USE CASE 1: DYNAMIC INCIDENT URGENCY BEHAVIOR (UI POLICY)

- * Condition: When Priority changes to 1 - Critical.
- * Action Matrix:
 - Short Description field becomes Read-Only.
 - Business Justification field becomes Visible and Mandatory.

USE CASE 2: DATE SEQUENCE VALIDATION (CLIENT SCRIPT)

- * Type: onSubmit Client Script
- * Logic: Fetches Planned Start Date and Planned End Date. If the end date is earlier than the start date, it halts the submission using return false; and displays an explicit error message on screen.

USE CASE 3: VIP USER HIGHLIGHT (CLIENT SCRIPT)

- * Type: onChange Client Script
- * Logic: When the Caller field changes, it checks if the user has a VIP flag set to true. If yes, it highlights the field in red or prints an info message using g_form.showFieldMsg().